

The standard library

Everything in this chapter arrives with the language. Nothing has to be installed, and nothing reaches the network to work.

The modules

Core

Module	For
<code>std.io</code>	<code>say</code> , <code>print</code> , <code>warn</code> , <code>ask</code> , <code>read_line</code> , <code>flush</code>
<code>std.math</code>	the usual functions, <code>pi</code> , <code>clamp</code> , <code>hypot</code> , <code>is_nan</code>
<code>std.time</code>	<code>now</code> , <code>clock</code> , <code>ms</code> , <code>sleep</code> , <code>format</code> , <code>parts</code>
<code>std.rand</code>	<code>seed</code> , <code>int</code> , <code>num</code> , <code>choice</code> , <code>shuffle</code> , <code>normal</code>
<code>std.sys</code>	<code>args</code> , <code>env</code> , <code>platform</code> , <code>arch</code> , <code>cpu_count</code> , <code>run</code> , <code>memory</code> , <code>exit</code>

Data

Module	For
<code>std.json</code>	<code>parse</code> , <code>write</code> , <code>pretty</code>
<code>std.csv</code>	<code>parse</code> , <code>write</code>
<code>std.hash</code>	<code>sha256</code> , <code>hmac_sha256</code> , <code>crc32</code> , <code>fnv64</code> , <code>base64</code>
<code>std.db</code>	the Ember database — chapter 19
<code>std.tensor</code>	numeric arrays — chapter 23

System

Module	For
<code>std.fs</code>	<code>read</code> , <code>write</code> , <code>append</code> , <code>exists</code> , <code>list</code> , <code>mkdir</code> , <code>size</code> , <code>copy</code>
<code>std.path</code>	<code>join</code> , <code>dirname</code> , <code>basename</code> , <code>ext</code> , <code>abs</code>
<code>std.thread</code>	<code>threads</code> , <code>locks</code> , <code>atomics</code> , <code>channels</code> — chapter 17
<code>std.task</code>	cooperative tasks

Networking, interface, pictures

Module	For
<code>std.net</code>	TCP and UDP sockets
<code>std.http</code>	<code>get</code> , <code>post</code> , <code>serve</code> , <code>url_encode</code> — chapter 22
<code>std.ui</code>	windows and controls — chapter 21

Written in SPRFST, in `std/`

Module	For
<code>std.core</code>	small helpers: <code>between</code> , <code>sign</code> , <code>blend</code> , <code>round_to</code> , <code>times</code> , <code>human_size</code> , <code>human_time</code>
<code>std.collections</code>	<code>zip</code> , <code>unique</code> , <code>chunks</code> , <code>tally</code> , <code>group_by</code> , <code>span</code> , <code>mean</code>
<code>std.testing</code>	<code>same</code> , <code>different</code> , <code>yes</code> , <code>no</code> , <code>close</code> , <code>holds</code> , <code>failed</code>
<code>std.ai</code>	layers, networks, tokenising — chapter 23

Open any of them: they are ordinary source files you can read.

A tour in one program

```
use std.io
use std.core
use std.collections
use std.math
use std.hash
use std.json

fn main() {
  io.say("- core -")
  io.say("between      {core.between(7.0, 1.0, 10.0)}")
  io.say("round_to      {core.round_to(3.14159, 2)}")
  io.say("blend         {core.blend(0.0, 100.0, 0.25)}")
  io.say("human_size     {core.human_size(1536000)}")
  io.say("human_time     {core.human_time(3725.0)}")

  io.say("")
  io.say("- collections -")
  let names = ["ada", "alan", "grace", "edsgar", "grace"]
  io.say("unique         {collections.unique(names)}")
  io.say("chunks         {collections.chunks([1, 2, 3, 4, 5], 2)}")
  io.say("zip            {collections.zip([1, 2, 3], ["a", "b", "c"])}")
  io.say("group_by       {collections.group_by(names, fn(n) => n.slice(0, 1))}")
  io.say("mean           {collections.mean([2.0, 4.0, 9.0])}")

  io.say("")
  io.say("- maths and hashing -")
  io.say("hypot           {math.hypot(3.0, 4.0)}")
  io.say("clamp           {math.clamp(42.0, 0.0, 10.0)}")
  io.say("sha256          {hash.sha256("sprfst").slice(0, 16)}...")
  io.say("base64          {hash.base64("sprfst")}")

  io.say("")
  io.say("- json -")
  let record = ["name": "notes", "version": 1, "tags": ["small", "fast"]]
  io.say(json.write(record))
}
```

Files and paths

```
use std.io
use std.fs
use std.path

fn main() {
    let file = "tour.txt"
    fs.write(file, "one\ntwo\nthree\n")
    fs.append(file, "four\n")

    let text = fs.read(file) ?? ""
    io.say("{text.trim().lines().len()} lines, {fs.size(file)} bytes")
    io.say("name {path.basename(file)} extension {path.ext(file)}")
    io.say("joined {path.join("notes", "today.md")}")

    fs.remove(file)
    io.say("gone: {not fs.exists(file)}")
}
```

Time

```
use std.io
use std.time

fn main() {
    let now = time.now()
    io.say("formatted {time.format(now, "%Y-%m-%d")}")
    let parts = time.parts(now)
    io.say("year {parts.get("year") ?? 0}, month {parts.get("month") ?? 0}")

    let started = time.clock()
    var total = 0
    for i in 0..50000 { total = total + i }
    io.say("counted to {total} in {(time.clock() - started) * 1000.0} ms")
}
```

`time.now()` is wall clock seconds since 1970 —the one to store. `time.clock()` is a monotonic clock — the one to measure with. `time.ms()` is milliseconds since the program started.

Finding out what exists

Three ways, none of which involve a web page:

```
sprfst docs .          # writes docs/ for your project and the library
```

In Studio, hover any name for its signature and its documentation line. And from any editor:

```
echo '{"id":1,"op":"natives"}' | sprfst studio
```

gives the whole native table —module, name, signature and summary —as JSON.

Exercise

Write a `report` program: read a CSV with `std.csv`, group the rows by their first column with `collections.group_by`, print a count per group and the total, and save the summary as JSON next to the input.